API Flows

Current edge and broker responsibilities

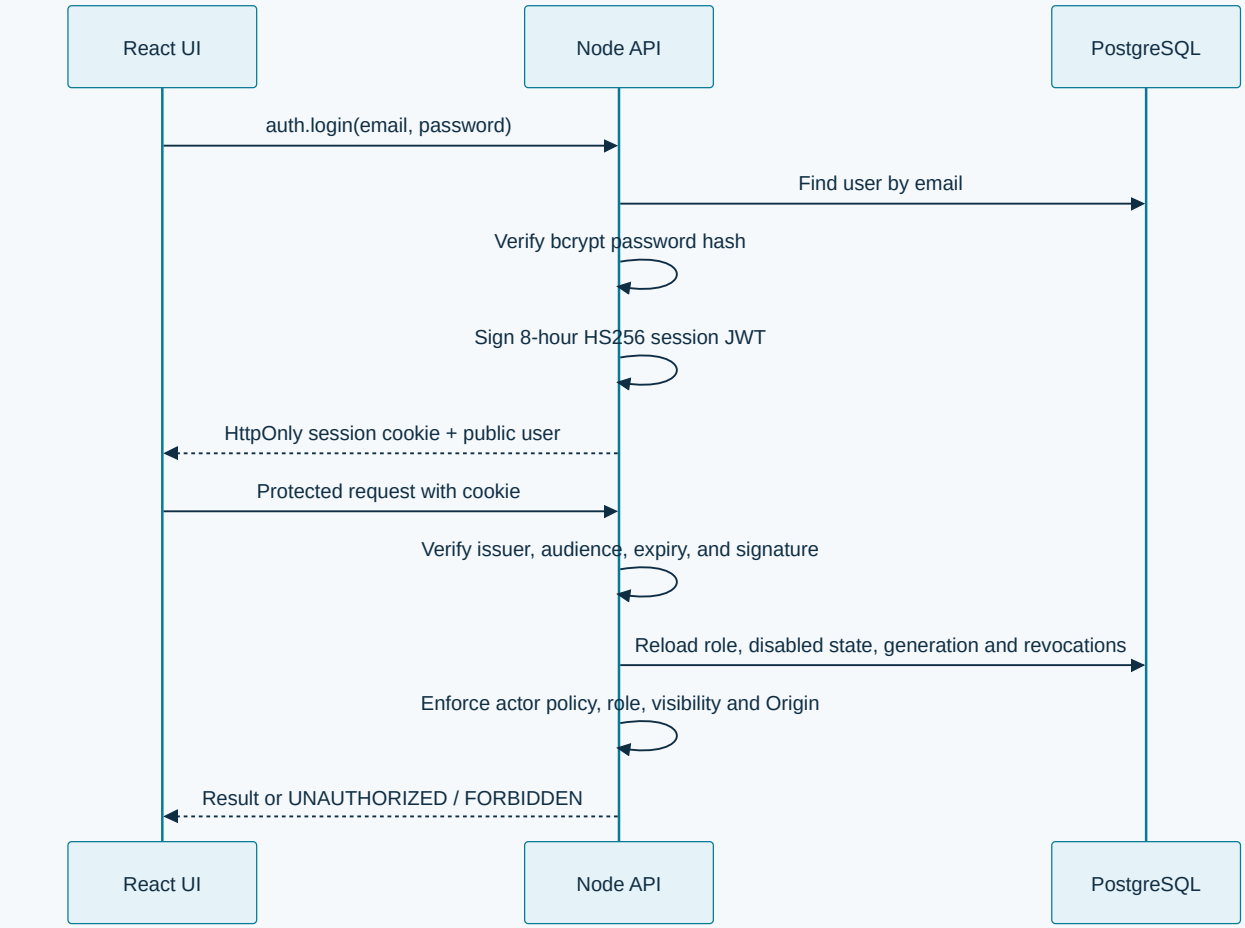
Source review: **6 November 2026**, canonical Smart Factory commit `52b4e4207cb7ceb27f419a3548887e63094d83af`. The current Raspberry Pi gateway guide specifies **local Mosquitto on the Pi and production Mosquitto in a separate Oracle VM container**. Older backend diagrams and staging runbooks contain superseded broker-provider references; the current edge guide and source-specific configuration take precedence. This documentation review is not a new inspection of the VM or physical commissioning.

Component	Responsibility and limits
Node-RED (optional)	Protocol/vendor adapters and transformations. Per-source commissioning and publish-only local broker ACLs; no cloud credentials or actuator authority. Existing native Python drivers remain available.
Python identity and validation	Check allowed source/topic, schema, types, units and timestamps. Stamp gateway provenance; reject spoofed integration claims and preserve unverified/bad quality until approved.
Python durable delivery	Persist accepted observations in SQLite before acknowledging eligible local messages; retry across restarts/outages with original timestamps. Remove upstream QoS 1/2 entries only after the corresponding broker handshake. This is not an end-to-end exactly-once guarantee.
Python command policy	Enforce authorized, commissioned targets and command expiry/replay protection. Safety interlocks and manual override stay at the machine. An acknowledgement is not physical feedback.
Python cybersecurity	Optional read-only SecurityCollector with an independent durable HTTPS path for identity, configuration, transport and certificate evidence. Disabled until its namespace/token is commissioned.
Python AI	Read-only AI advisory capability is in active development. Development status does not establish enabled production inference; any AI component remains subject to source authorization and command policy.
Oracle Mosquitto and worker	TLS, dedicated credentials and scoped topics. The .NET worker independently persists/validates telemetry and retries application-API delivery. Redis is transient fan-out, not the durable telemetry record.

Direct MQTT devices need separate credentials, network reachability and device ACLs. The current Pi-only broker policy does **not** enable them. Node-RED and Python do not replace safety-rated controller functions. See Node-RED commissioning and Python security/AI scope.

The React client calls the Node/Express tRPC API. Server-side procedures enforce the user's current database role; client-side route and button checks only control presentation.

Login and authorization



Login and authorization · Open the diagram for a full-size view.

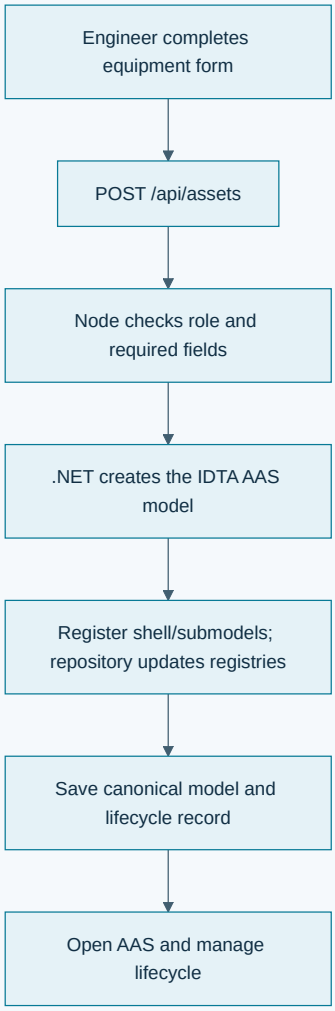
Public registration submits details over HTTPS; the API stores a password hash in `account_requests` and creates no account or session. An individual administrator verifies the applicant, chooses a role, and approves or rejects the request. The sign-in page's **Forgot password?** form gives the same confirmation for every address. For an eligible account with configured email delivery, the API stores a hashed, expiring token and an encrypted message for the existing mail worker. The email link opens a new-password form; the browser removes the token from the URL and retains it only in memory. Successful redemption consumes the token, changes the password and invalidates previous sessions without signing the user in automatically. Administrator-assisted one-use temporary credentials remain available. The browser neither reads nor stores the session JWT. All protected requests check persisted role, disabled state, session generation and revocation. Reserved demo identities and individuals selecting Demo read only approved server fixtures; editable simulations stay in the browser. Actor and workspace checks precede operational writes, exports and service delegation. See account access.

Register and delete connectivity devices

Engineers and administrators can call `devices.create` with `type: "gateway" | "edge_device"`, a stable device ID, name, and optional location/zone. Older callers that omit `type` default to `gateway`. Only administrators can call `devices.delete`. A gateway linked to an AAS must be disconnected first; telemetry history is retained.

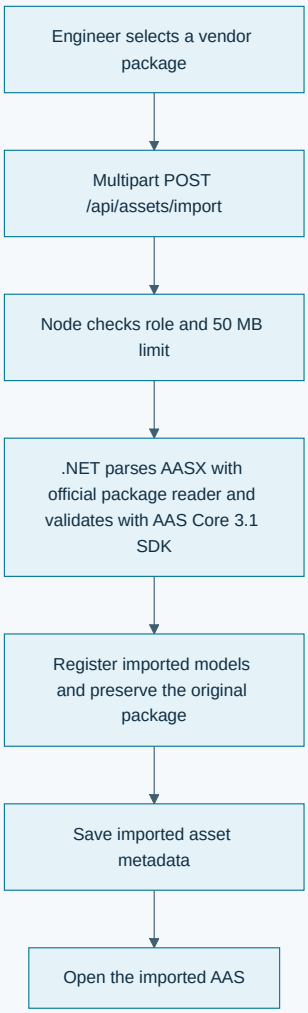
Register an asset and open its AAS

Quick Create (Form)



Quick Create (Form) · Open the diagram for a full-size view.

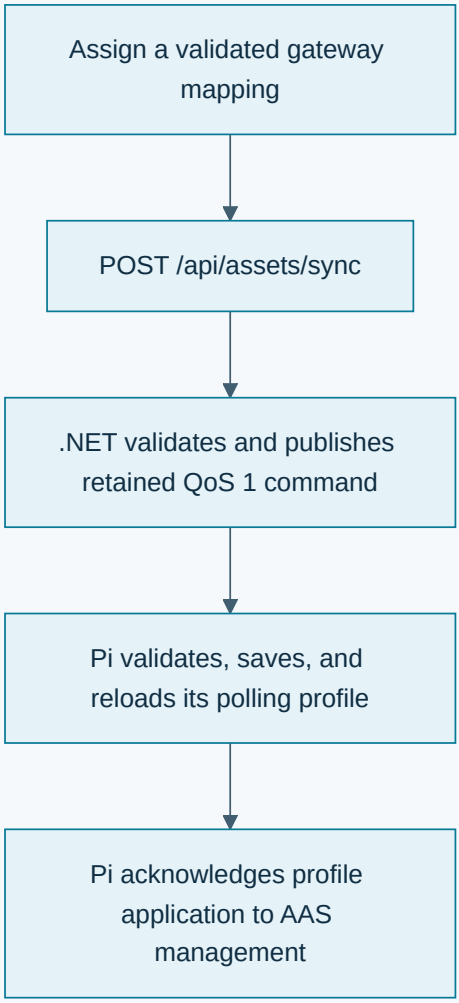
Import Package (.aasx)



Import Package (.aasx) · Open the diagram for a full-size view.

Form-managed AAS edits submit the revision currently loaded by the engineer. PostgreSQL serializes changes per asset, the .NET service confirms the matching repository revisions before replacing them, and the API appends a snapshot plus SHA-256 digest. A stale update returns HTTP 409. Restoring a snapshot creates a new revision. Reusing an imported shell identifier is rejected by the unique asset key and does not replace the existing package.

Deploy and confirm an edge profile



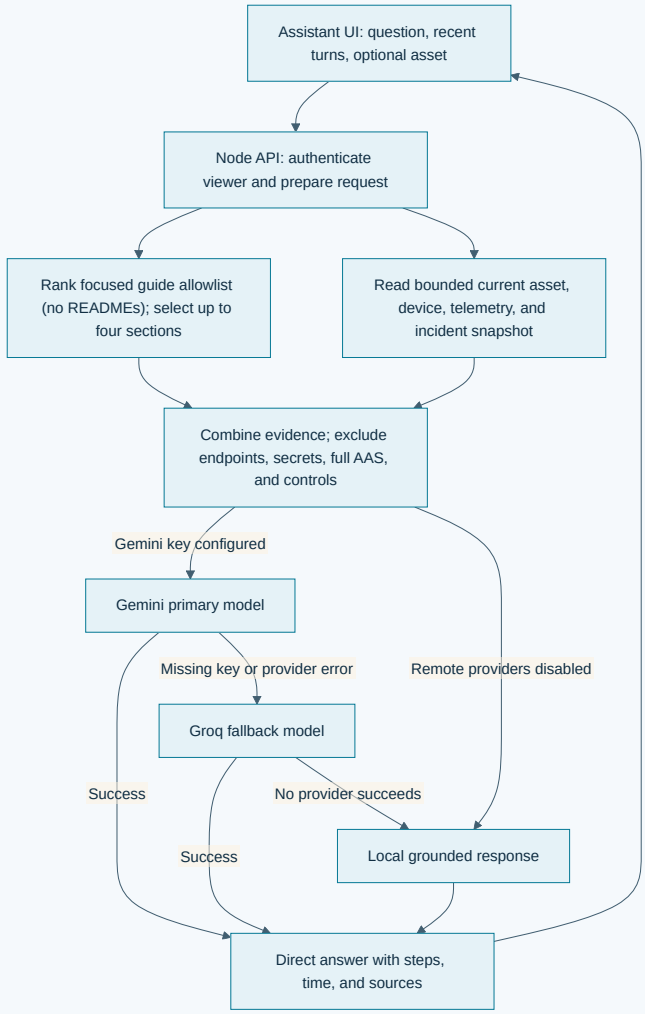
Deploy and confirm an edge profile · Open the diagram for a full-size view.

The UI's `assets.*` tRPC procedures expose dashboard workflows. `/api/assets/import` is a role-checked multipart API with a 50 MB package limit. `/api/aas/*` is the standards API gateway: the Node service checks the application session and engineer/admin role, resolves an OAuth token endpoint from OIDC discovery or uses its configured direct URL, and forwards the request with a service bearer token. Writes to dashboard-managed records are blocked at this gateway so edits cannot skip revision history. Do not expose AAS services directly to browsers.

After a repository mutation, DeviceService publishes an `aas_changed` event to Redis Pub/Sub. Each dashboard replica fans the event out to its own authenticated WebSocket clients, so a browser connected to any replica can refresh its AAS view. Engineers and admins can export the current asset hierarchy as CAEX 3.0 from **Assets → Export AutomationML**; the export carries stable asset IDs and basic metadata but not gateway mappings.

Only administrators can call `assets.delete`. It removes the selected dashboard AAS and its represented BaSyx resources, retains historical telemetry and incidents, and reports gateway resynchronization failures.

Assistant context and provider routing



Assistant context and provider routing · Open the diagram for a full-size view.

The `assistant.ask` procedure is available to signed-in viewers and higher. It ranks only the explicit focused-guide allowlist in `server/assistantKnowledge.ts` (README files are excluded), adds a bounded snapshot from the dashboard database, and sends recent conversation context plus relevant evidence to Gemini first and Groq second when server API keys are configured. Keys remain on the server. Answers are formatted for readability and include ordered steps for how-to questions; the UI shows context capture time and source excerpts without provider/model labels or a provider data-transmission notice. With providers disabled or unavailable, a local grounded response remains available. The Assistant does not read environment files, README files, full AAS packages, private account records, or machine-control interfaces. See Assistant configuration and data boundaries.

Factory layouts, local demo edits and diagnostics

`factoryLayouts.list/get/save/archive` checks the current actor, owner and `live/configured` workspace scope before reading or changing layout documents. Saving includes the expected version; conflicting updates cannot overwrite a newer layout. A layout changes visualization only. It does not move machinery, change telemetry attribution or commission an asset. Shared demo sessions keep drafts in browser storage and can download them as JSON; they cannot save shared server layouts. See factory layouts.

The editable demo sandbox overlays bounded browser-local simulated changes on approved fixtures. Reset and JSON backup/import affect that sandbox. Requests that would mutate operational inventory, shared fixtures, control hardware or send real notifications retain their server authorization checks. Assistant requests may include a bounded local simulated snapshot; it is labelled simulated and is answered locally rather than sent to an external provider. See Assistant.

For FX/TSN, `networkDiagnostics.overview` returns authorized diagnostics or explicit demo scenarios. An individual engineer/admin may use `networkDiagnostics.importSnapshot`; the commissioned service collector instead uses `POST /api/internal/network-diagnostics` with dedicated gateway-scoped credentials. Both validate the same bounded report schema before replacing the latest gateway snapshot. Import and collector ingestion are read-only with respect to PLCs and network schedules. Export downloads the selected snapshot. See FX/TSN report contract, retention and diagram.

Asset Analytics sends selected AAS identifiers and a selected time range to `analytics.getAssetTelemetry`; readings are selected by their persisted asset attribution and can be scoped to one asset or filtered into metadata groups. The response includes per-asset means and peaks for power, temperature, vibration, pressure, and speed; linked-device alert counts; alert events in the period; and early-to-recent temperature, vibration, and pressure changes where the observed sample span supports comparison. These changes are descriptive screening indicators: they are not degradation models, failure predictions, or remaining-useful-life estimates. Supported source units are converted to validated canonical units; unknown legacy units remain unavailable for aggregates. Descriptive comparisons retain per-asset context. The service does not calculate OEE, capacity utilization, kWh, energy per unit, emissions, causal root cause, TCO, or MTBF until its required production, calibrated energy, maintenance, failure, and cost inputs are recorded.

The **Dashboard** summarizes API-backed asset, gateway, lifecycle, active-alert, and confirmed downtime state, including the longest current outage and average downtime-to-resolution. **Live Monitoring** shows gateway and edge-device connectivity with batched latest readings. **Alerts** combines the response queue and searchable/exportable records, while threshold configuration remains attached to gateway details.

Gateway publication updates a complete desired-state profile over MQTT. QoS 1 broker publication is not an edge-application receipt; the Pi publishes an acknowledgement under the matching command topic. The operator may also download the profile for controlled offline deployment. Credentials are configured on the edge host and are not included in either path.

Edge configuration, control, and firmware

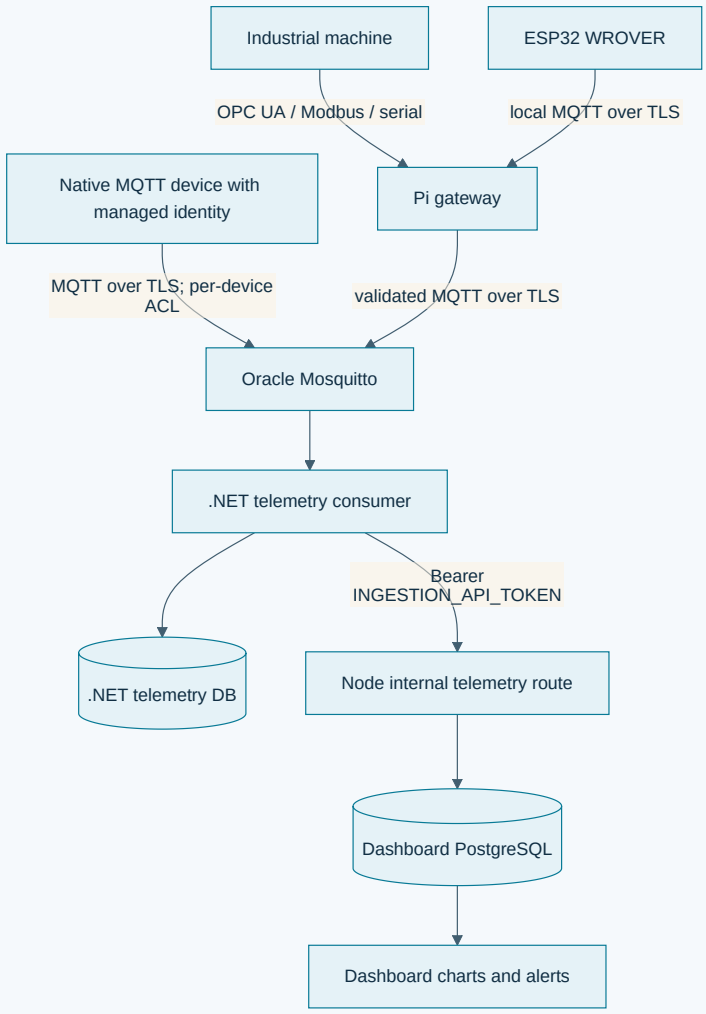
These are separate operations. **Edge configuration** sends the gateway a desired-state profile containing asset identity, protocol mappings, and telemetry rules; only the gateway's matching application acknowledgement indicates that it applied the profile. **Control** sends a bounded, device-specific command through an authorized protocol adapter. **Firmware update** replaces executable code and requires a compatible updater, trusted release artifact, integrity/signature verification, device-reported result, and recovery strategy. A configuration acknowledgement does not verify firmware or machine motion.

Target	Configuration / control path	Firmware update path in this release
Raspberry Pi gateway	MQTT desired-state profile with gateway acknowledgement; supported adapters are documented in the edge repository.	Manual staged deployment over SSH/rsync; no remote updater or dashboard OTA.
ESP-WROVER-KIT	Telemetry and bounded GPIO18 pulse requests route through the Pi; operators and higher can use the dashboard's commissioned indicator control. A GPIO acknowledgement confirms a logic output only.	Build with PlatformIO and flash over USB; firmware has no OTA receiver.
ADA031 V4 arm	USB serial through the Pi gateway; the AAS page exposes bounded joint jogs for authorized roles. No position feedback or independent safety function is claimed.	Program the Uno over USB using the vendor sketch; no OTA.
Other industrial assets	Requires an asset-specific protocol adapter, authorization policy, interlocks, limits, and verified feedback appropriate to that machine.	Requires a vendor-supported update mechanism and tested release/recovery process; there is no universal industrial OTA protocol.

The firmware inventory page is read-only. Backend OTA submission/status endpoints return `501 Not Implemented` and do not mutate rollout state until a verified release service and authenticated device agent exist. Legacy rollout rows are historical application records, not proof of installation. Do not use software commands as emergency-stop, guarding, or other protective functions.

Edge telemetry

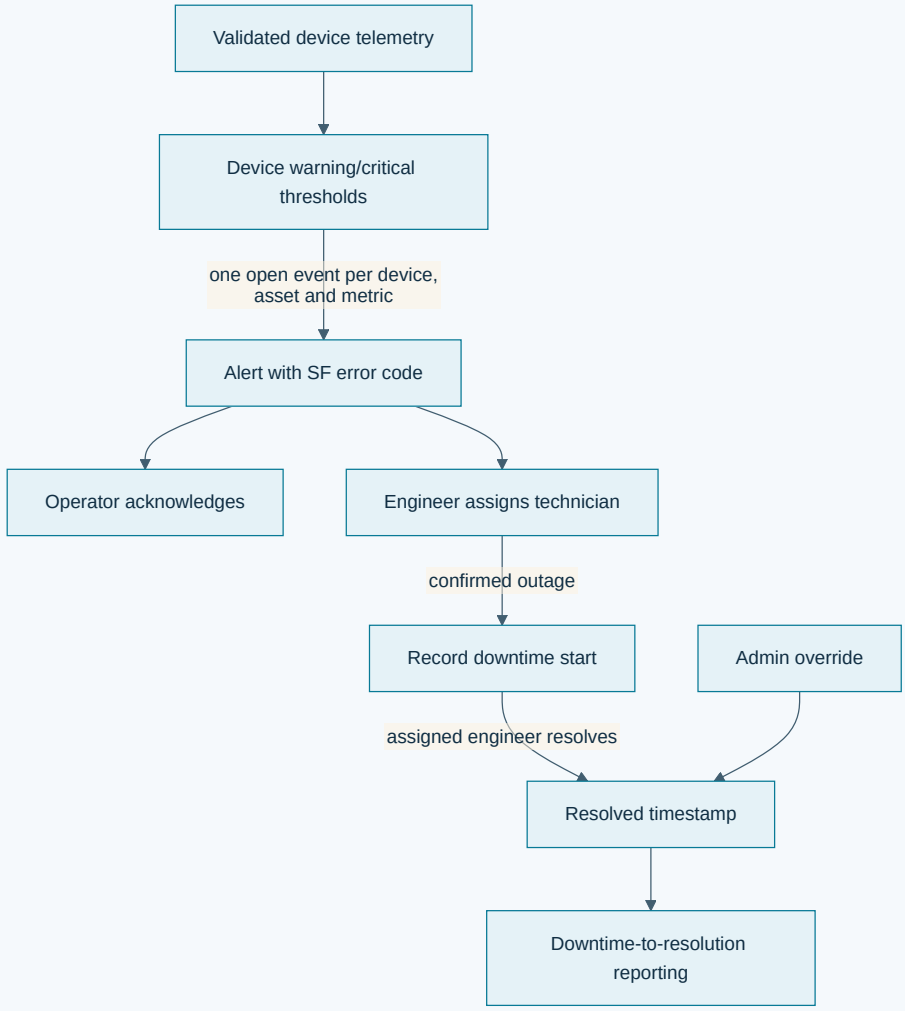
Connected ADA031 firmware emits controller state, cycle count, commanded targets, and movement-active signals while a program runs. The Pi publishes these as `assetSignals`, and the AAS view renders their latest values. After USB re-enumeration the gateway discards the failed serial descriptor and reopens the stable `/dev/serial/by-id/...` path on the next poll. This is controller telemetry, not measured physical joint feedback.



Edge telemetry · Open the diagram for a full-size view.

WROVER and ADA031 remain behind the Pi gateway. The WROVER is allowlisted by the Pi and can publish only to the Pi-local Mosquitto listener; its firmware has no Oracle Mosquitto credentials or internet route. The direct-device software path requires separate provisioning and is not enabled by the current Pi-only broker policy. The gateway validates its local topic, identity, payload size, timestamp, and metric fields before forwarding. Normalized timestamps are Unix epoch milliseconds in UTC. The authenticated telemetry bridge preserves the observed route separately and validates it against the commissioned source relationship. Delayed or duplicate samples never reparent devices. Complete canonical IDs remain copyable in item details. The bridge validates the payload and compares a service-only token in constant time. This credential is independent of user sessions and must only be installed in backend service configuration.

Alert generation and resolution



Alert generation and resolution · Open the diagram for a full-size view.

Alert codes are platform classifications unless a source-specific code is provided. A threshold event does not automatically imply machine downtime; the assigned engineer-role incident responder records that point when the machine is confirmed unavailable. Device-offline alerts report lost communication; they do not automatically start factory downtime. Historical inferred downtime records require review. Only alerts with a downtime start and a resolution time contribute to the mean downtime-to-resolution metric. Operators can acknowledge, while assigned engineer-role responders or admins can resolve.

See authorization and AAS for the role matrix, template versions, configuration variables, and verification commands.

Identity, coverage and incident notifications

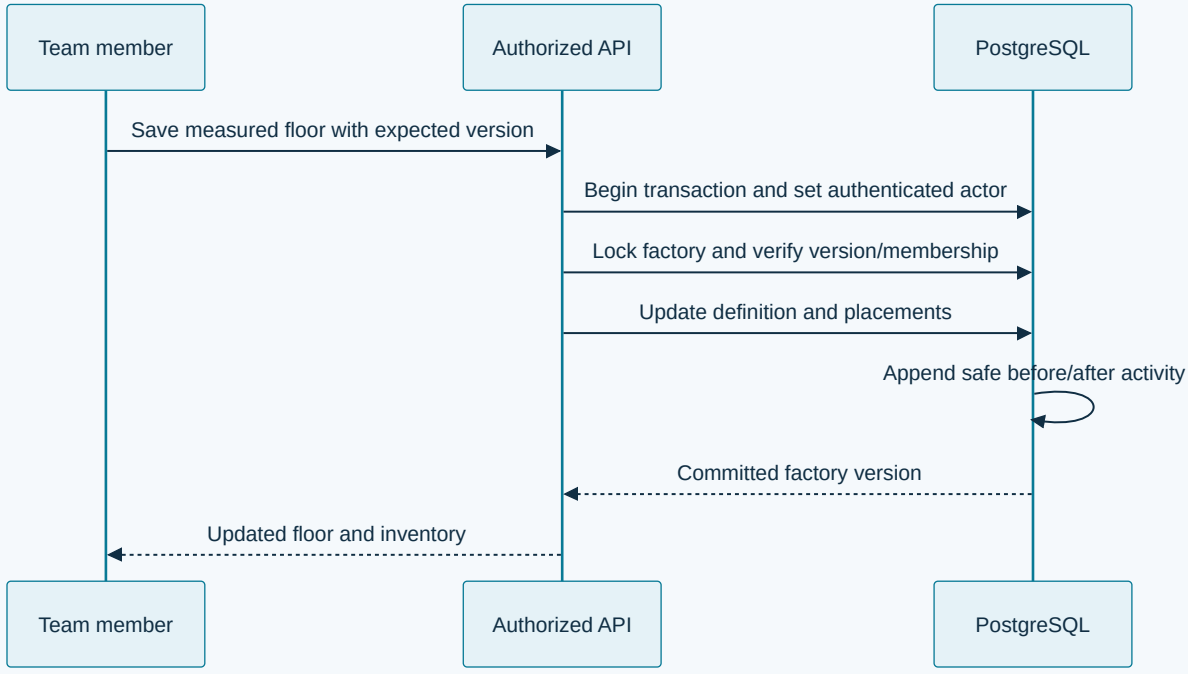
In the Oracle split bundle, `auth.me` delegates to the private IdentityService with the authenticated dashboard account ID and a separate service token; the service reads the current role from PostgreSQL. It never returns an email-provider or AAS access token. `analytics.getCoverage` delegates a bounded selected-asset/time range to AnalyticsService, which aggregates indexed stored samples without treating missing metrics as zero.

Alert writes atomically create the per-user inbox via the database trigger. Account creation queues a password-free welcome message without rolling back the new account if queueing is temporarily unavailable. `notifications.list` and `notifications.markRead` are authenticated owner-only operations. `notifications.retryEmail` requires an

engineer/admin owner and an eligible failed/unconfigured/unauthorized incident job. Reading a notification is independent of incident acknowledgment. The UI links incident notifications to event details by ID. NotificationService delivers the durable queue through the configured Resend sender and checks current account role and recipient policy at delivery time.

Shared named factories and history

`factories.list/get/available` provide authorized team factories and paged unassigned operational inventory. `create/save/add/attach` require Engineer; `deleteItem` requires Admin, the exact factory membership and current version. A layout save cannot add/remove membership. New equipment starts planned and unconfigured, and existing registration completes its AAS publication explicitly. Every inventory mutation uses the existing canonical records.



Shared named factories and history · Open the diagram for a full-size view.

`factories.activity.list` pages fifty server-attributed entries using a cursor; `activity.get` returns the selected safe before/after detail for the factory. There are no activity update/delete routes. Shared demo sessions cannot write these endpoints; the browser sandbox adapter supplies clearly simulated factory edits/history. Assistant requests carry an optional active factory identity, keeping evidence scoped to that factory. See named factories.